[Nom de l’établissement]
Cycle ingénieur — Informatique

Rapport de fin d’études

Conception et réalisation d’une plateforme de marketplace multi-acteurs
ASMAD Marketplace

Réalisé par : [Prénom NOM]

Encadrant(e) académique : [Prénom NOM]

Entreprise / contexte professionnel (si applicable) : [À compléter ou N/A]

Année universitaire : 2024 — 2025 · **Période projet :** janvier — juin 2025

> **Export PDF :** ouvrir ce fichier dans Chrome ou Edge, attendre le rendu des diagrammes (quelques secondes), puis *Fichier → Imprimer → Enregistrer au format PDF*. Cocher « Graphiques d'arrière-plan » si les couleurs de tableaux sont pâles.

Table des matières

1. Résumé
2. Introduction générale
3. Contexte, objectifs et périmètre
4. Analyse des besoins
5. Méthodologie de gestion de projet (Scrum)
6. Planification : épics, sprints et user stories
7. Architecture technique et choix technologiques
8. Conception objet et modélisation UML
9. Sécurité, qualité et contraintes
10. Tests, validation et Definition of Done
11. Conclusion et perspectives
12. Bibliographie et webographie
13. Annexes

Résumé

Ce rapport présente la démarche d'ingénierie menée pour concevoir et développer **ASMAD Marketplace**, une application web de type marketplace mettant en relation des **clients**, des **vendeurs** et un **administrateur**. Le travail couvre l'analyse des besoins, la planification selon une approche **Agile Scrum** sur six mois (janvier à juin 2025), l'architecture logicielle en couches (front-end **Vue.js**, API **NestJS**, base **MySQL**), ainsi que la modélisation UML (diagrammes de classes et cas d'utilisation) et des scénarios détaillés pour les flux métiers majeurs.

Mots-clés : *marketplace, e-commerce, Scrum, NestJS, Vue.js, TypeORM, JWT, UML, ingénierie logicielle, rapport de fin d'études.*

Introduction générale

Les plateformes de commerce en ligne occupent une place centrale dans l'économie numérique. Elles exigent à la fois une **expérience utilisateur** fluide pour les acheteurs, des **outils de gestion** fiables pour les vendeurs, et un **pilotage** par les administrateurs. Le présent projet s'inscrit dans une logique de **projet d'ingénierie complet** : du recueil du besoin à la livraison incrémentale d'un prototype exploitable.

L'objectif pédagogique est double : (1) démontrer la maîtrise du **cycle de vie logiciel** en contexte agile, et (2) appliquer les principes de **conception orientée objet**, de séparation des responsabilités et de sécurité des échanges client-serveur. Le document est structuré comme un rapport de fin d'études type **ingénieur informatique** : chaque section apporte à la fois une **justification** (pourquoi), une **description** (quoi) et un lien avec la **réalisation** (comment).

1. Contexte, objectifs et périmètre

1.1 Contexte

ASMAD Marketplace vise à offrir un **catalogue produit** consultable par le grand public, des **espaces métiers** distincts (vendeur, administrateur) et un parcours d'**achat** (panier, commande). Le contexte technique est celui d'une application **SPA** (Single Page Application) dialoguant avec une API REST sécurisée.

1.2 Problématique

Comment organiser le développement d'une application web multi-rôles, tout en garantissant la **traçabilité des exigences**, la **livraison par incréments** testables, et une **base de données cohérente** avec les règles métier (stock, commandes, avis, relations vendeur-client) ?

1.3 Objectifs

- **Fonctionnels :** authentification et gestion des rôles ; administration des catégories et utilisateurs ; gestion des produits et médias par le vendeur ; panier et commandes ; avis et modération ; suivi de vendeurs.
- **Non fonctionnels :** sécurisation des API (JWT, contrôle d'accès), maintenabilité (modularisation NestJS / composants Vue), évolutivité modérée du modèle de données.

1.4 Périmètre et hors périmètre (exemple)

Dans le périmètre	Hors périmètre (exemple indicatif)
Authentification, rôles, CRUD métier cœur, UML et rapport	Passage en production industrialisée, PCI-DSS complet pour carte bancaire, application mobile native

2. Analyse des besoins

2.1 Acteurs du système

Acteur	Rôle
Visiteur	Consultation du catalogue sans obligation de compte (selon règles déployées).
Client	Compte acheteur : panier, commande, avis, wishlist, suivi de vendeurs.
Vendeur	Gestion de l'offre (produits, catégories vendeur, commandes associées, avis reçus).

Acteur	Rôle
Administrateur	Supervision globale : utilisateurs, catégories, marchés, commandes, modération des commentaires.
Système externe	Service de messagerie (e-mail) pour la réinitialisation de mot de passe.

2.2 Exigences fonctionnelles (extrait)

ID	Exigence	Priorité
EF-01	S’inscrire, se connecter, réinitialiser le mot de passe	Haute
EF-02	Isoler les espaces admin / vendeur / client par contrôle d’accès	Haute
EF-03	Gérer catégories, produits, images	Haute
EF-04	Panier persistant et création de commande	Haute
EF-05	Avis produits, consultation vendeur, suppression admin	Moyenne
EF-06	Suivre des vendeurs (relation client–vendeur)	Moyenne

2.3 Exigences non fonctionnelles (extrait)

ID	Exigence	Indicateur / moyen
ENF-01	Sécurité des accès API	JWT, validation des rôles côté serveur
ENF-02	Intégrité des données	Contraintes BDD, transactions sur commande
ENF-03	Maintenabilité	Modules NestJS, composants Vue réutilisables
ENF-04	Expérience utilisateur	Layouts par rôle, retours visuels (toasts, formulaires)

3. Méthodologie de gestion de projet (Scrum)

3.1 Pourquoi Scrum ?

Le développement d'une marketplace comporte de nombreuses **incertitudes** (priorisation des écrans, affinage des règles métier, retours utilisateurs). La méthode **Scrum** privilégie des **cycles courts** (sprints), une **inspection** régulière du produit (review) et une **adaptation** du backlog (planning / refinement). Elle est adaptée à un contexte de **projet étudiant ou startup** où la vélocité doit être ajustée semaine après semaine.

3.2 Rôles (adaptation projet)

Rôle Scrum	Correspondance type PFE
Product Owner	Définit la vision, priorise le backlog (tuteur / porteur métier / l'étudiant en autonomie guidée).
Scrum Master	Facilite les rituels, lève les blocages (souvent combiné au développeur en petit groupe).
Équipe de développement	Réalisation front, back, tests, documentation.

3.3 Artefacts et événements

- **Product Backlog :** liste ordonnée d'épics et user stories.
- **Sprint Backlog :** sous-ensemble engagé pour le sprint (2 semaines).
- **Incrément :** version du logiciel potentiellement livrable en fin de sprint.
- **Rituels :** planification de sprint, daily courte, revue de sprint, rétrospective.

3.4 Calendrier global

La période retenue est **janvier à juin 2025**, soit environ **26 semaines**, découpées en **12 sprints de 2 semaines** (avec une marge intégrée dans la dernière release pour stabilisation et rédaction).

4. Planification : épics, sprints et user stories

4.1 Stratégie de release

Trois **épics majeurs** matérialisent trois **releases** logiques : d'abord les **fondations** (sans lesquelles aucun parcours n'est sécurisé), puis l'**offre et le back-office**, enfin l'**achat et la confiance** (panier, commande, avis, social vendeur).

Release	Sprints	Objectif synthétique
R1 — Fondations	S1 à S2	Environnement, persistance, authentification, routage par rôle.
R2 — Offre & back-office	S3 à S6	Administration, catalogue vendeur, produits, médias, vitrine lecture.
R3 — Achat & confiance	S7 à S12	Panier, commandes, avis, modération, suivi vendeurs, profils, finitions UX.

4.2 Détail des user stories par sprint (vue rapport)

Chaque user story est formulée selon le format *« En tant que …, je veux …, afin de … »*. La **Definition of Done** (section 8) s'applique systématiquement.

Épic R1 — Sprints 1 et 2

US	Énoncé	Valeur métier
US1.1	En tant qu'équipe, je veux des builds reproductibles et une configuration d'environnement documentée, afin de réduire les écarts « ça marche sur ma machine ».	Industrialisation légère du PFE.
US1.2	En tant que système, je veux un modèle utilisateur persisté (MySQL + ORM), afin de supporter l'inscription et la connexion.	Base métier.
US1.3	En tant qu'utilisateur, je veux m'authentifier par JWT, afin d'accéder à mon espace selon mon rôle.	Sécurité et personnalisation.
US1.4	En tant qu'utilisateur, je veux réinitialiser mon mot de passe par e-mail, afin de récupérer l'accès en autonomie.	Réduction du support.
US1.5	En tant qu'utilisateur, je veux des layouts et gardes de navigation par rôle, afin d'éviter les accès non autorisés côté UI.	UX et conformité accès.

Épic R2 — Sprints 3 à 6

US	Énoncé	Valeur métier
US2.1	En tant qu'admin, je veux gérer les catégories globales, afin de structurer le catalogue.	Référentiel produit.
US2.2	En tant qu'admin, je veux gérer vendeurs et clients, afin de contrôler les comptes.	Gouvernance plateforme.
US2.3	En tant qu'admin, je veux gérer les marchés, afin de refléter l'organisation commerciale.	Alignement organisationnel.
US2.4	En tant que vendeur, je veux gérer mes catégories et mes produits avec images, afin de publier mon offre.	Cœur du modèle économique vendeur.
US2.5	En tant que visiteur, je veux consulter catalogue et fiche produit, afin d'évaluer l'achat.	Acquisition et conversion.
US2.6	En tant qu'admin/vendeur, je veux un tableau de bord synthétique, afin de suivre l'activité.	Aide à la décision basique.

Épic R3 — Sprints 7 à 12

US	Énoncé	Valeur métier
US3.1	En tant que client, je veux un panier persistant, afin de composer mon achat sur plusieurs visites.	Conversion.
US3.2	En tant que client, je veux passer commande (checkout), afin de finaliser l'achat.	Transaction.
US3.3	En tant que vendeur/admin, je veux consulter les commandes selon mes droits, afin d'assurer le suivi logistique.	Exécution de la vente.
US3.4	En tant que client, je veux publier un avis, afin d'informer la communauté.	Confiance / preuve sociale.
US3.5	En tant qu'admin, je veux modérer les avis, afin d'appliquer la charte de bonne conduite.	Qualité du contenu.
US3.6	En tant que client, je veux suivre des vendeurs et gérer ma wishlist, afin de personnaliser mon expérience.	Fidélisation.
US3.7	En tant qu'utilisateur, je veux gérer mon profil, afin de tenir mes informations à jour.	Qualité des données.
US3.8	En tant qu'équipe, je veux internationalisation / thème / messages homogènes, afin de livrer un produit « fini » pour soutenance.	Qualité perçue.

5. Architecture technique et choix technologiques

5.1 Vue d'ensemble

L'architecture retenue est une **architecture en couches découplées** :

- **Couche présentation :** application Vue (SPA), routage côté client, consommation d'API REST.
- **Couche application / domaine :** modules NestJS (auth, user, product, order, etc.) encapsulant la logique et les contrôleurs HTTP.
- **Couche persistance :** MySQL via TypeORM ; entités et relations traduisant le modèle métier.

5.2 Justification des choix

Technologie	Rôle	Justification courte
Vue.js	UI réactive	Courbe d'apprentissage raisonnable, écosystème mature pour formulaires et vues métier.
NestJS	API structurée	Architecture modulaire proche des bonnes pratiques back-end enterprise (injection, modules).
TypeORM	ORM	Mapping objet-relationnel explicite, migrations / synchronize en phase projet.
MySQL	SGBDR	Modèle relationnel adapté aux transactions commande et intégrité référentielle.
JWT	Auth stateless	Adapté à une SPA ; nécessite discipline sur expiration et contrôle serveur systématique.

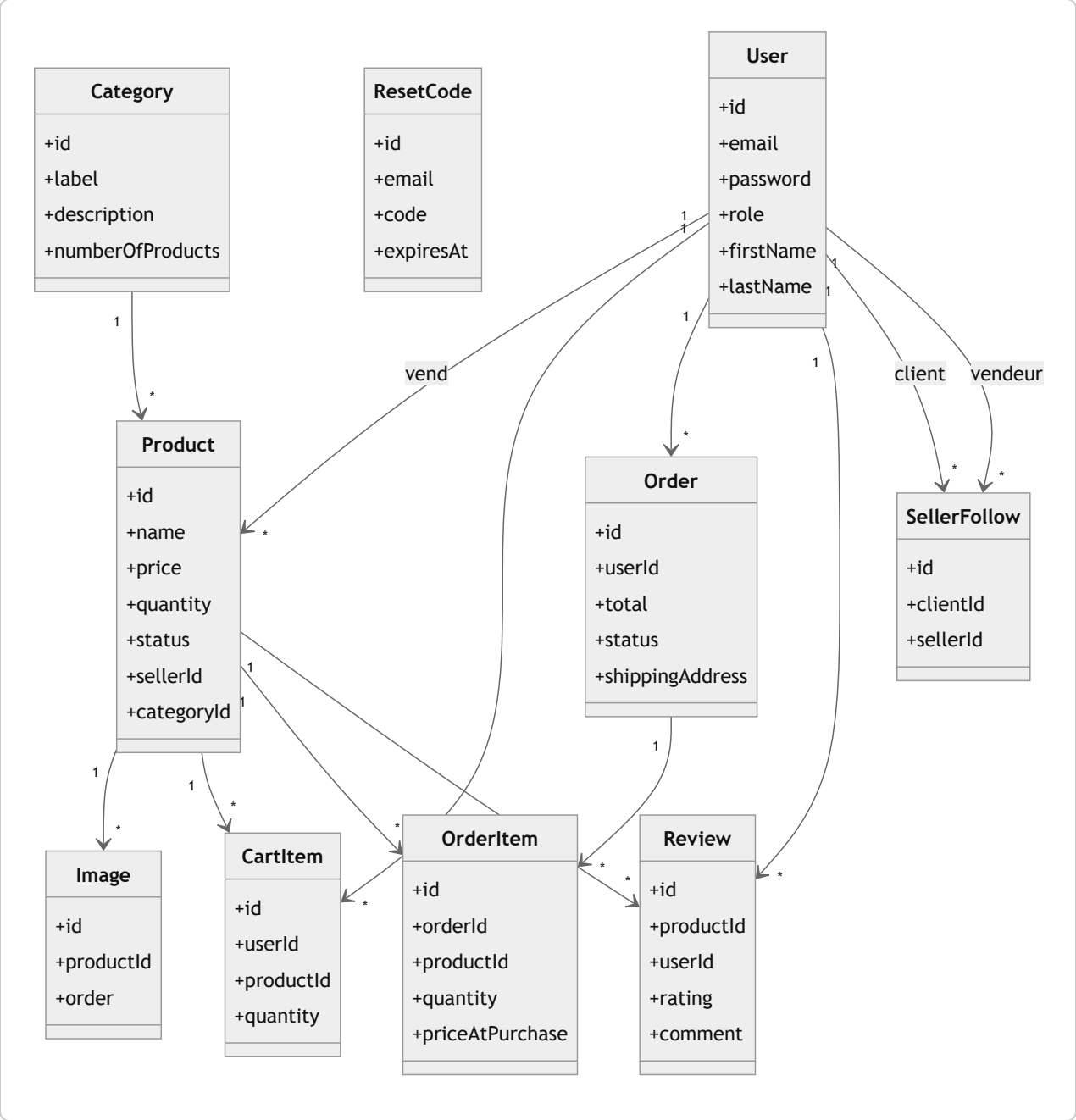
5.3 Principes appliqués

On privilégie la **séparation des responsabilités** (contrôleurs fins, services métier), la **validation des entrées** (DTO, pipes NestJS), et le **principe du moindre privilège** pour les endpoints sensibles (admin, vendeur).

6. Conception objet et modélisation UML

6.1 Diagramme de classes global — interprétation

Le diagramme suivant synthétise les **entités persistées** et leurs associations principales. Il sert de **vue d'ensemble** pour la cohérence du domaine : un `User` peut être vendeur de plusieurs `Product` ; un `Product` appartient à une `Category` ; le panier (`CartItem`) lie utilisateur et produit ; la commande (`Order`) agrège des `OrderItem` avec un **prix figé** (`priceAtPurchase`) pour l'historique comptable ; les `Review` et `SellerFollow` matérialisent l'engagement client.



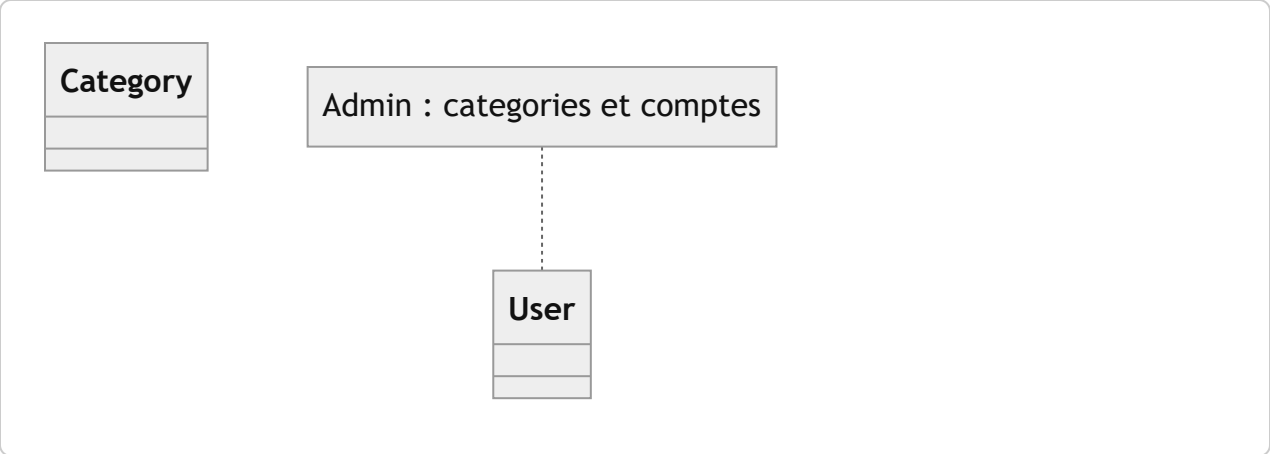
6.2 Raffinement par sprint (incréments de modèle)

En méthode agile, le modèle **n'est pas figé le premier jour** : les sprints ajoutent des concepts. Les diagrammes ci-dessous illustrent **quel sous-graphe du domaine est concerné** à chaque phase, ce qui facilite la traçabilité « story → classes impactées » dans un mémoire.

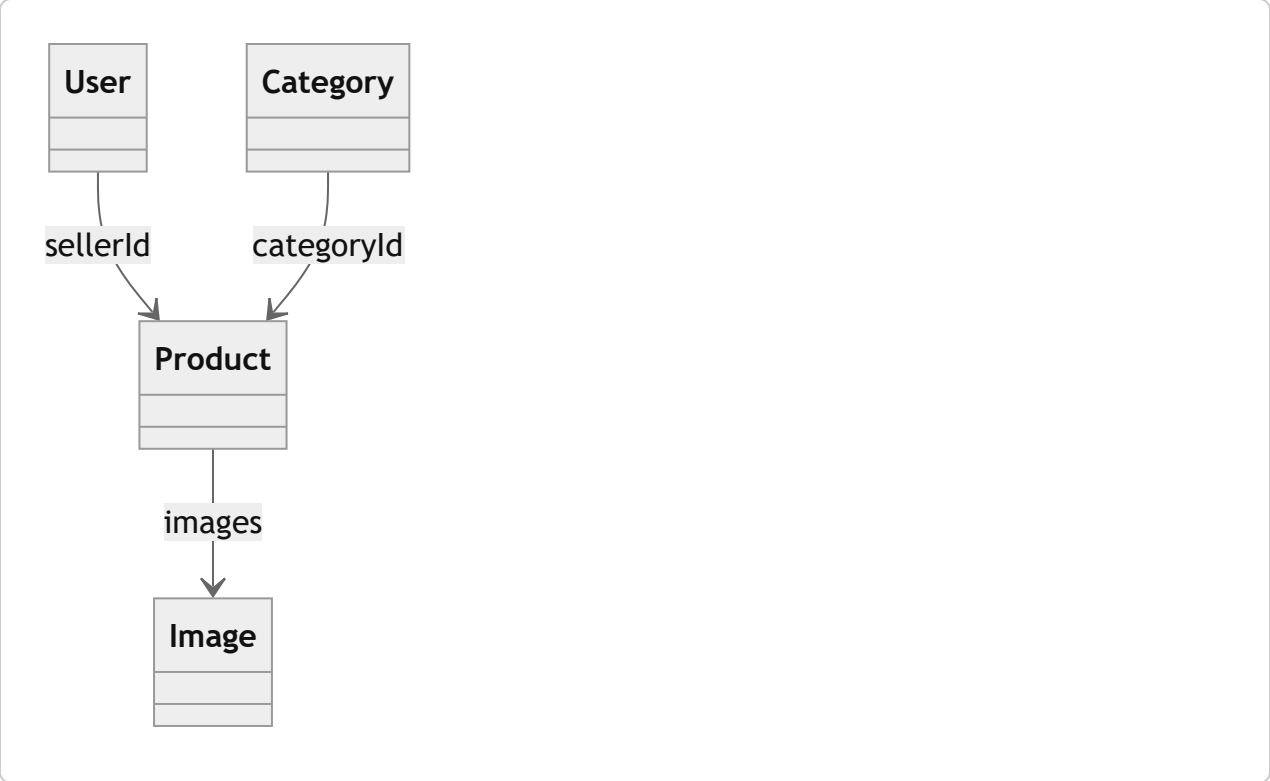
Sprints 1–2 — Fondation identité

Commentaire ingénieur : la séparation `User` / `ResetCode` évite de surcharger l'entité utilisateur avec des attributs temporaires ; le code possède une durée de vie et une politique d'expiration.

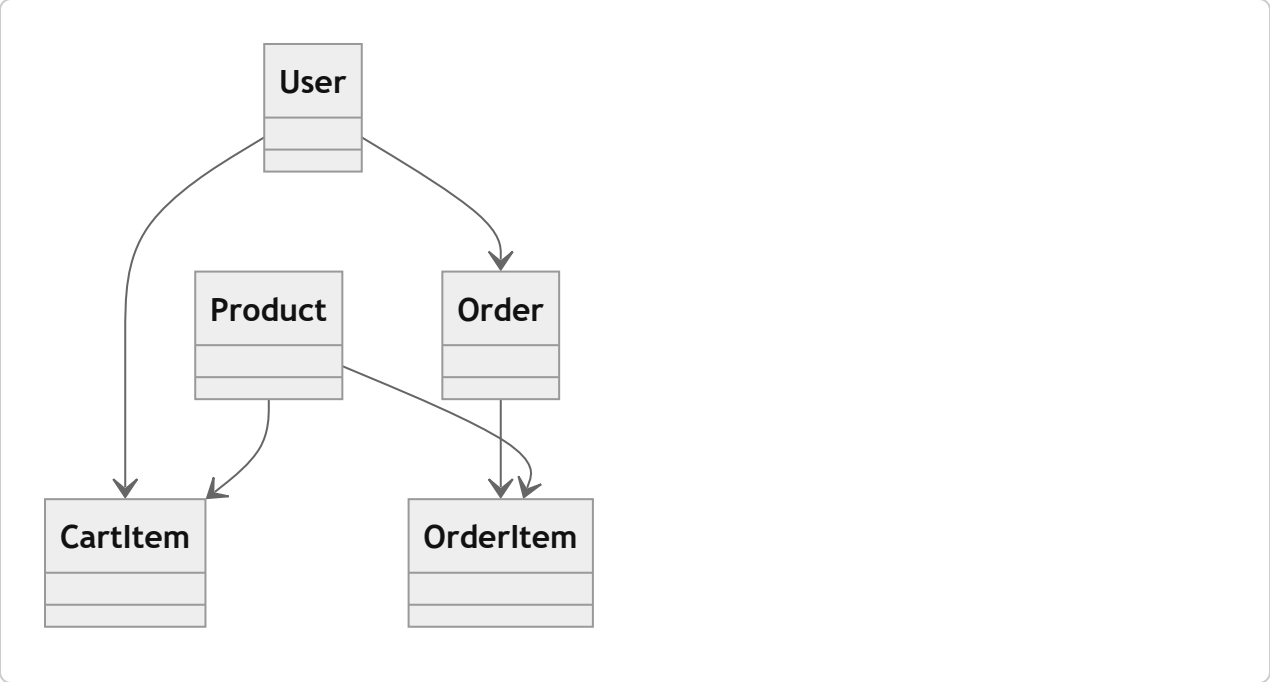
Sprints 3–4 — Référentiel et administration



Sprints 5–6 — Offre catalogue vendeur

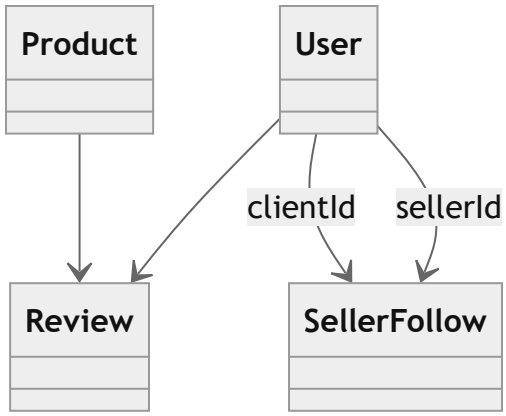


Sprints 7–8 — Transaction



Remarque : dupliquer le prix dans `OrderItem` est une décision de conception courante pour conserver l'historique même si le prix catalogue du produit change ensuite.

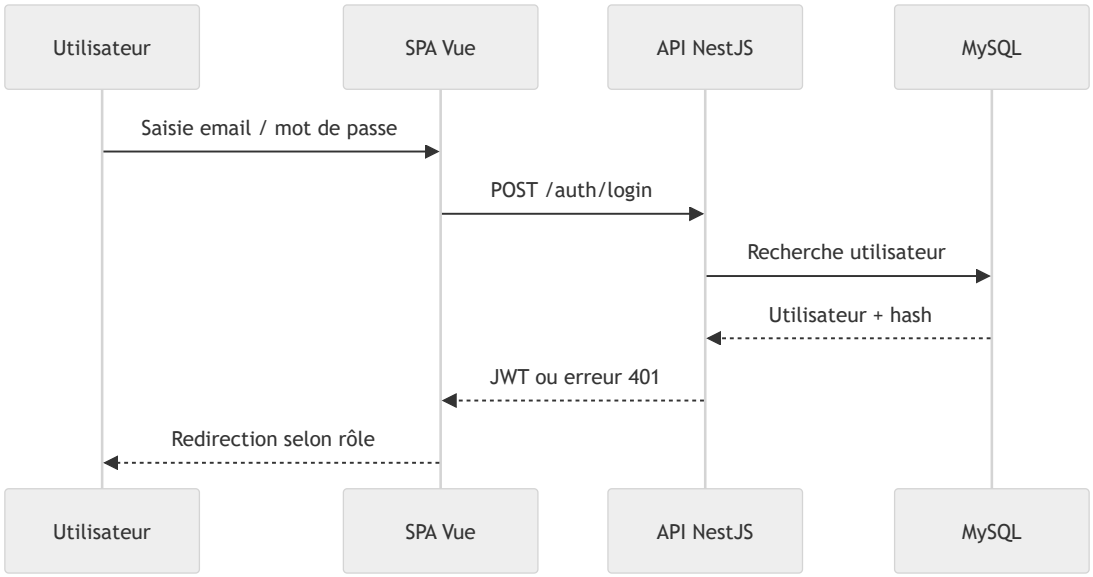
Sprints 9–12 — Confiance et relation client–vendeur



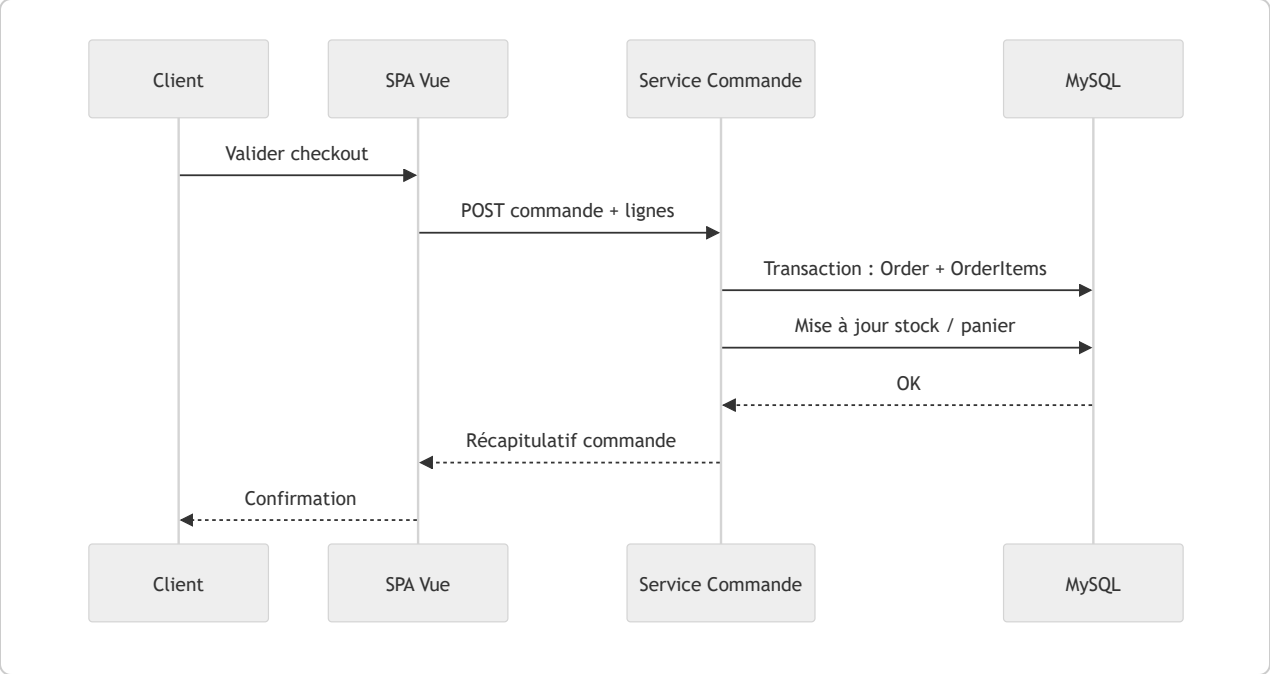
6.3 Diagrammes de séquence (flux techniques majeurs)

Les diagrammes de séquence complètent les cas d'utilisation en montrant les **interactions temporelles** entre acteur, front-end, API et base de données.

Connexion avec JWT (simplifié)



Création de commande depuis le panier (simplifié)



6.4 Cas d'utilisation — périmètre et descriptions détaillées

Les diagrammes suivants situent les acteurs par rapport au système. Pour chaque cas, le tableau précise les **scénarios nominal et alternatifs**, conformément aux bonnes pratiques de rédaction de cahier des charges logiciel.

UC1 — Inscription et connexion

Acteurs	Visiteur, Système d'authentification.
Préconditions	Inscription : e-mail non utilisé. Connexion : compte existant et actif.
Scénario nominal (connexion)	1) Saisie des identifiants. 2) Vérification serveur (hash du mot de passe). 3) Émission d'un JWT signé et daté. 4) Stockage côté client (ex. localStorage) et application des gardes de route. 5) Redirection vers l'espace *client*, *vendeur* ou *admin*.
Scénario alternatif A	Mot de passe incorrect : code HTTP 401, message utilisateur sans divulguer si l'e-mail existe (bonne pratique anti-énumération, selon niveau de sécurité retenu).

Scénario alternatif B	Compte inexistant : proposition d'inscription ou message générique.
Postconditions	Session authentifiée cohérente avec le rôle, ou échec sans effet de bord sur les données métier.

UC2 — Réinitialisation du mot de passe

Scénario nominal	1) Demande avec e-mail. 2) Génération d'un code à durée de vie limitée, persistance `ResetCode`. 3) Envoi asynchrone par e-mail. 4) Saisie du code et du nouveau mot de passe. 5) Mise à jour du hash, invalidation ou purge du code.
Alternatif	Code expiré ou erroné : affichage d'erreur, possibilité de renouveler la demande dans la limite du throttling anti-abus (recommandation).

UC3 — Gestion des produits (vendeur)

Scénario nominal	1) Authentification vendeur. 2) Création / édition des attributs (nom, description, prix, stock, statut, catégorie). 3) Association d'images ordonnées. 4) Persistance transactionnelle des entités `Product` et `Image`. 5) Rafraîchissement des vues liste et détail.
Règle métier	Seul le vendeur propriétaire (ou l'admin selon politique) peut modifier ses produits ; le contrôle doit être **redondant** côté API.

UC4 — Consultation catalogue et fiche produit

Syntax error in text
mermaid version 10.9.5

Scénario nominal	1) Requête liste avec filtres / pagination selon implémentation. 2) Sélection d'un produit. 3) Affichage des médias, prix (éventuellement remisés), disponibilité, lien vers la boutique du vendeur.
Postconditions	Lecture seule ; pas de modification d'état métier obligatoire.

UC5 — Panier et commande

Syntax error in text
mermaid version 10.9.5

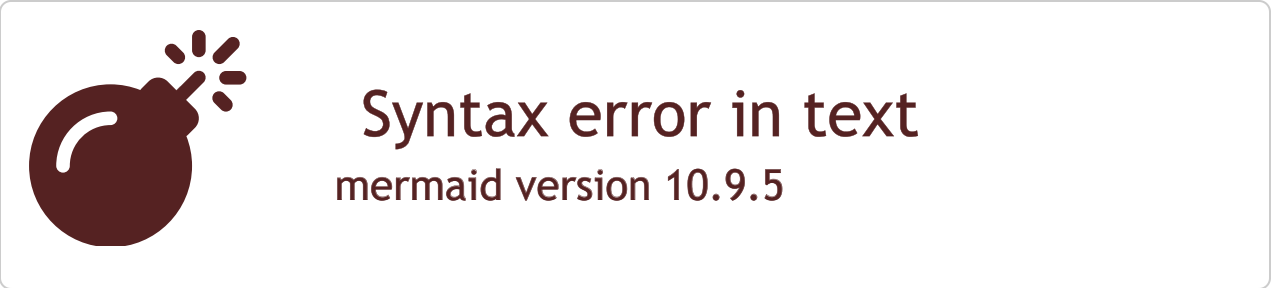
Scénario nominal	1) Ajout d'une ligne panier (`CartItem`) ou incrément de quantité. 2) Vérification de cohérence avec le stock. 3) Checkout : collecte adresse et mode de paiement (selon périmètre). 4) Création atomique de `Order` et `OrderItem` avec prix figés. 5) Mise à jour des stocks et nettoyage du panier.
Alternatif	Stock insuffisant : message utilisateur, rollback ou ajustement de ligne.

UC6 — Suivi des commandes

Syntax error in text
mermaid version 10.9.5

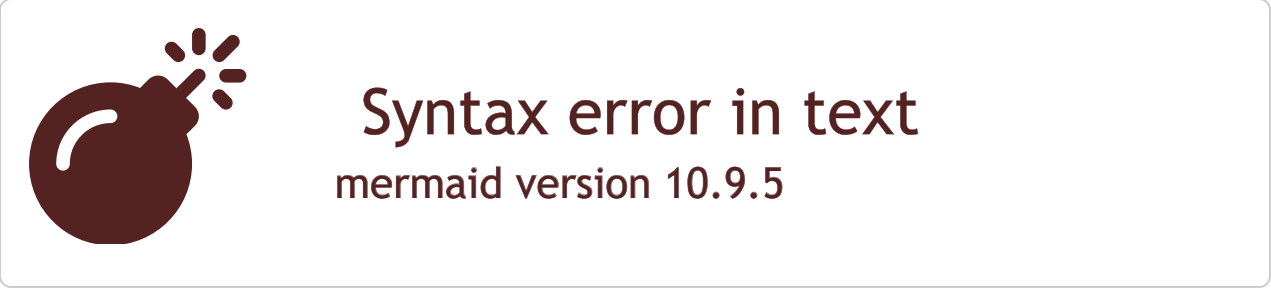
Scénario nominal (vendeur)	Filtrage des commandes contenant au moins un produit du vendeur connecté ; consultation des lignes ; mise à jour de statut si prévue.
Exigence de sécurité	Aucune fuite de données commande entre vendeurs ; l'admin dispose d'une vue transverse légitime.

UC7 — Avis et modération



Scénario nominal (client)	Saisie note + commentaire, création `Review` liée au couple (utilisateur, produit) avec règles d'unicité ou de temporisation possibles.
Scénario nominal (admin)	Consultation des avis (ex. agrégation par vendeur) ; suppression avec confirmation UX et appel API sécurisé.

UC8 — Suivre un vendeur



Scénario nominal	Création ou suppression d'une ligne `SellerFollow` respectant la contrainte d'unicité (clientId, sellerId).

7. Sécurité, qualité et contraintes

7.1 Sécurité

- **Authentification stateless** par JWT : chaque requête sensible doit être validée côté serveur (signature, expiration).
- **Autorisation par rôle** : les décorateurs / guards NestJS doivent refléter exactement les mêmes règles que la navigation Vue (defense in depth).
- **Mots de passe** : stockage hashé (jamais en clair) ; politique de complexité recommandée en annexe projet.
- **Données sensibles** : ne pas exposer d'informations internes dans les messages d'erreur destinés au client.

7.2 Qualité logicielle

La qualité est traitée par des **revues de code**, des **tests manuels systématiques** en fin de sprint, et l'usage d'un **Definition of Done** partagé. L'objectif n'est pas seulement « coder la fonctionnalité », mais **réduire la dette technique** (duplication, couplage fort front–back).

7.3 Contraintes

Contraintes typiques d'un PFE : temps limité, équipe réduite, dépendance aux services externes (mail), et nécessité de documenter pour la **soutenance** plutôt que d'optimiser prématurément la performance à très grande échelle.

8. Tests, validation et Definition of Done

8.1 Stratégie de test

Niveau	Objet	Exemples
Tests manuels exploratoires	Parcours utilisateur	Inscription, login, checkout, droits vendeur/admin.
Tests API (optionnel avancé)	Contrats REST	Postman / tests automatisés Jest sur contrôleurs critiques.
Tests de régression légers	Stabilité sprint	Checklist avant démo de fin de sprint.

8.2 Definition of Done (équipe projet)

Une user story est considérée comme terminée lorsque :

1. Le code est intégré sur la branche principale d'intégration et a été relu (pair review ou auto-review structurée).
2. Les scénarios d'acceptation associés à la story ont été exécutés avec succès (preuve : capture d'écran, note de test, ou démo en review).
3. Les contrôles d'accès côté API sont en place pour les nouveaux endpoints.
4. Aucune anomalie bloquante ouverte sur la story ; les anomalies mineures sont soit corrigées soit reportées avec justification.
5. La documentation minimale est à jour (README, variable d'environnement, ou commentaire d'API si pertinent).

9. Conclusion et perspectives

Ce rapport a présenté la démarche complète menée pour ASMAD Marketplace : **analyse des besoins**, **planification agile** sur six mois, **architecture** moderne SPA + API + SGBDR, et **modélisation UML** appuyée sur des scénarios détaillés. Les livrables obtenus démontrent la capacité à structurer un projet d'ingénierie informatique de bout en bout.

Perspectives : intégration d'un **passerelle de paiement** certifiée, couverture de tests automatisés plus large, observabilité (logs centralisés, métriques), passage en **CI/CD** cloud, et durcissement sécurité (OWASP ASVS, limitation de débit sur l'auth). Ces axes constituent une feuille de route réaliste au-delà du cadre strict du mémoire.

10. Bibliographie et webographie

- Schwaber, K. & Sutherland, J. — *The Scrum Guide* (version en vigueur à la date du projet).
- Documentation officielle : Vue.js, NestJS, TypeORM, MySQL.
- Fowler, M. — *Patterns of Enterprise Application Architecture* (concepts de couche service, repository).
- OWASP — guides applicables (Top 10, sécurisation API).

Annexes

Annexe A — Glossaire

Terme	Définition
JWT	JSON Web Token : jeton signé transportant des claims d'identité ; utilisé pour authentifier les requêtes API.
ORM	Object-Relational Mapping : couche faisant correspondre classes et tables relationnelles.
SPA	Single Page Application : application web dont la navigation est gérée côté client.
DoD	Definition of Done : critères communs de finition d'une user story.

Annexe B — Liste des figures UML du rapport

Figure 1 : diagramme de classes global. Figures 2 à 6 : raffinements par sprint. Figures 7 et 8 : diagrammes de séquence (connexion, commande). Figures 9 à 16 : diagrammes de cas d'utilisation et tableaux de scénarios associés.

Document rédigé pour un rapport de fin d'études — modèle à compléter (noms, logos d'établissement, captures d'écran de l'application en annexe si exigé par votre école). Fichier source : `ASMAD-Rapport-Fin-Etudes.html`.